

The Sliced Sandwich Construction

A two-sided keyed slicing mechanism on $2n$ wires

local_mixing project

July 20, 2026; balanced variant September 4, 2026

branch `sbg-gen-mix-clean`

Abstract

The *sliced sandwich* is a keyed obfuscation construction that computes a source function C on one designated input slice and, on the same slice, reveals only a fresh random function D^{-1} when run backwards. It is a slicing mechanism specific to the sandwich structure and is distinct from the gadget and feistel paths: it uses no secret sharing, placing C and a random padding computation D as plaintext g57 circuits on the low half of $2n$ wires, with two slice blocks of CNOT/CCNOT gates interleaved through the two computations. It comes in two variants, selected by a choice bit: the original *classic* layout, in which both computations share the low half and both directions read out on the high one, and a *balanced* layout, in which each half hosts one computation and each direction has its own slice and its own answer half. This note gives both constructions, their slice semantics in both directions, the rationale, and the parameters.

1 Wires and structure

The transformed circuit acts on $2n$ wires in two halves of n : the first half (wires $0..n-1$) holds the input x , and the second half (wires $n..2n-1$) holds the auxiliary y — the *slice register*.

The circuit is a sandwich of two computations around a copy step:

$$A = [C \parallel S_1] ; N ; [D \parallel S_2],$$

where $\parallel$ denotes a random interleaving. The pieces:

- C — the source circuit, a g57 circuit placed directly on wires $0..n-1$. No secret sharing: this is a plaintext computation whose obfuscation comes from the subsequent mixing.
- D — a fresh random circuit of m g57 gates on wires $0..n-1$, the *same design* as the C block (target and two distinct controls per gate). Default $m = n(\log_2 n)^2$.
- N — the copy step $y \hat{=} x$: the n CNOTs wire $(n+i) \hat{=} \text{wire } i$. (This is the classic direction; see §3.)
- S_1, S_2 — two independent slice blocks of s gates each (default $s = n \log_2 n$). Every gate targets the first half and reads at least one second-half wire with positive polarity:

$$\underbrace{x_i \hat{=} y_j}_{\text{CNOT, } \approx 1/3} \quad \underbrace{x_i \hat{=} x_j \wedge y_k}_{\text{CCNOT, rest}}$$

so each block is *dead when the second half is zero*. S_1 is randomly interleaved with C , S_2 with D ; the interleaving is uniform and preserves each computation's internal order (necessary for C and D to compute correctly).

2 Slice semantics (classic)

2.1 Forward: the slice reveals C

On the zero slice $y = 0$, trace the running state (a, b) from $(x, 0)$:

$$(x, 0) \xrightarrow{C \parallel S_1} (C(x), 0) \xrightarrow{N} (C(x), C(x)) \xrightarrow{D \parallel S_2} (\text{junk}, C(x)).$$

During the first stage the second half is still 0, so every S_1 gate is dead and C computes correctly. The copy step moves $C(x)$ into the second half. In the third stage S_2 is *live* (the second half now holds $C(x) \neq 0$), but it only targets the first — junk — half, so it cannot corrupt the answer. Hence

$$\boxed{A(x, 0) = (\text{junk}, C(x))}$$

with the answer on the second half.

2.2 Inverse: the same slice reveals D^{-1}

Every gate is an involution, so A^{-1} is the reversed gate list. On input $(p, 0)$:

$$(p, 0) \xrightarrow{(D \parallel S_2)^{-1}} (D^{-1}(p), 0) \xrightarrow{N} (D^{-1}(p), D^{-1}(p)) \xrightarrow{(C \parallel S_1)^{-1}} (\text{junk}, D^{-1}(p)).$$

Now it is S_2 that is dead (its stage runs first, second half 0), so D^{-1} computes cleanly; and S_1 that fires (its stage sees a nonzero second half), junking the first half. Hence

$$\boxed{A^{-1}(p, 0) = (\text{junk}, D^{-1}(p)).}$$

The construction is thus sliced on the *same* slice in both directions: each slice block guards one direction's correctness (S_1 forward, S_2 inverse) and scrambles the other direction's junk. The forward oracle gives C ; the backward oracle gives only the random D^{-1} , never C^{-1} .

2.3 Off-slice

For $y \neq 0$ the second-half output is $y \oplus C'(x, y)$, where C' is the S_1 -disturbed computation. The $y \oplus$ masking from the copy step guarantees the output differs from the clean answer $C(x)$ on every nonzero slice, independently of whether the disturbance C' happens to coincide with C at a given x .

3 The balanced variant

The *balanced* variant changes two things and leaves everything else — C , S_1 , the interleaving, the float stage — untouched:

- the copy step flips direction, $N : x \hat{=} y$ (wire $i \hat{=} \text{wire } (n+i)$), and

- the D block moves to the second half, D' on wires $n..2n-1$.

A third change is forced by those two: S_2 must *mirror* with D , targeting the second half and reading the first,

$$\underbrace{y_i \hat{=} x_j}_{\text{CNOT, } \approx 1/3} \quad \underbrace{y_i \hat{=} y_j \wedge x_k}_{\text{CCNOT, rest}}$$

so that S'_2 is dead when the *first* half is zero. Otherwise S_2 would fire as soon as D' makes the second half nonzero, and would overwrite the forward answer, which in this layout stays in the first half. So

$$A = [C \parallel S_1] ; N(x \hat{=} y) ; [D' \parallel S'_2].$$

3.1 Forward: the slice reveals C on the first half

On the same zero slice $y = 0$, from $(x, 0)$:

$$(x, 0) \xrightarrow{C \parallel S_1} (C(x), 0) \xrightarrow{N} (C(x), 0) \xrightarrow{D' \parallel S'_2} (C(x), \text{junk}).$$

Block 1 is unchanged, so S_1 is dead and C computes correctly; the flipped copy step is then a no-op on the slice; and block 2 writes *only* the second half, so the answer is frozen where block 1 left it:

$$\boxed{A(x, 0) = (C(x), \text{junk})}$$

with the answer on the first half. Where the classic variant protects the answer by moving it into a register that the second block never targets, the balanced variant protects it by leaving it in a half that the second block never targets.

3.2 Inverse: the mirrored slice reveals D^{-1}

The inverse is sliced at $x = 0$, not $y = 0$. On input $(0, q)$:

$$(0, q) \xrightarrow{(D' \parallel S'_2)^{-1}} (0, D^{-1}(q)) \xrightarrow{N} (D^{-1}(q), D^{-1}(q)) \xrightarrow{(C \parallel S_1)^{-1}} (\text{junk}, D^{-1}(q)).$$

S'_2 is dead because the first half is zero, so D^{-1} computes cleanly on the second half; N copies it down; and the reversed block 1 — C^{-1} with S_1 now firing on a nonzero second half — junks the first half only. Hence

$$\boxed{A^{-1}(0, q) = (\text{junk}, D^{-1}(q)).}$$

3.3 Off-slice

For $y \neq 0$ the first-half output is $C'(x, y) \oplus y$ with C' the S_1 -disturbed computation: the flipped copy step supplies exactly the same $y \oplus$ masking as the classic one, now on the answer half, so no nonzero slice yields the clean answer.

3.4 What is balanced, and what it costs

Each half now hosts one computation and is one direction's payload: the classic layout's low half is a shared workspace and its high half a pure answer register, whereas here C owns the first

half and D the second. The two directions consequently have *different* slices ($y = 0$ forward, $x = 0$ backward), where the classic construction slices both at $y = 0$.

Equivalently: the classic layout has a half — the low one — that is junk in *both* directions, and the balanced layout has none. One asymmetry comes with it: on the forward slice the D' lane starts from the constant 0 rather than from live data, so its junk output is a function of x only through the S'_2 injections, whereas the classic layout’s junk half starts from $C(x)$ and runs D on it.

4 Rationale

Why interleave rather than prepend. In the plain gadget path the slice block sits at the input port. Here the sandwich has two computations and a natural two-sided structure, and weaving each slice block *through* its computation ties the disturbance to intermediate states rather than only the boundary, which the mixing then disperses. Correctness is unaffected because the slice gates are dead on the slice regardless of their positions.

Why D mirrors C . D is drawn from the same distribution as the source blocks (random g57 gates on the low n wires) so that the forward and inverse directions are structurally indistinguishable: an adversary holding the mixed circuit sees two same-design computations and cannot tell from gate statistics which end carries the “real” function.

Why this gate vocabulary. The slice gates are positive-polarity CNOT/CCNOT — the same mixing-native vocabulary as the rest of the circuit, with no complemented gates and no polarity pattern that could betray the slice. The slice is the canonical zero vector, so nothing about it is encoded in the circuit beyond the deadness structure.

5 Parameters

symbol	meaning	default
n	wires of the source function (circuit is $2n$ wires)	—
m	gates in the random D computation	$n(\log_2 n)^2$
s	gates in each slice block S_1, S_2	$n \log_2 n$
variant	classic or balanced (§3)	classic

Requires $n \geq 3$ (g57 gates need three distinct wires). Invocation:

```
sss --cnot --sliced-sandwich -n <n> --sandwich-m <m> --sandwich-s <s> [--sandwich-balance
...
```

The variant is chosen per run, never per build, and defaults to `classic` everywhere. For `gen_sandwich_gadget` it is the final positional argument or the environment variable `SANDWICH.VARIANT` (an explicit positional wins). The transformed size before mixing is $|C| + m + n + 2s$ gates (source, D , copy step, two slice blocks); e.g. for $n = 8$ with a 40-gate source and defaults ($s = 24$, $m = 72$): $40 + 72 + 8 + 48 = 168$ gates, of which $40 + 72 = 112$ are g57.

6 Verification

The driver checks the pinned contract after transformation and after every mixing round: the **SandwichSecond** view for the classic variant (the second half of the input is forced to zero and the second-half output must equal $C(x)$) and the **SandwichFirst** view for the balanced one (same pinned slice, first-half output must equal $C(x)$). Unit tests cover slice-gate deadness on both sides, each variant’s forward slice contract with a permutation check, the classic inverse’s second-half bijection on the slice, the balanced inverse’s exact recovery of D^{-1} on the mirrored slice, and the float stage’s band for both variants.

Provenance. Code: `SandwichVariant`, `sliced_sandwich_cnot`, `sliced_sandwich_with_d`, `sliced_sandwich_build`, `random_g57_xgates`, `sandwich_slice_gates`, `shift_xgate_wires`, `random_interleave` in `src/preprocessing/gadgets.rs`; driver branch and `FunctionView::SandwichSecond` / `FunctionView::SandwichFirst` in `src/db_mixing/main_mix_cnot.rs`; flags in `src/main.rs` and `src/commands/sss.rs`; variant argument in `src/preprocessing/bin/gen_sandwich_gadget.rs`. Classic circuits are unchanged: the choice bit defaults to `classic` and the builder draws the same values in the same order as before, so a classic seed still reproduces its circuit bit for bit. Companion quick-reference: `docs/SLICED_SANDWICH.md`.